

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris



The Adoption of Python Array API in scikit-learn

by Stefanie Senger and Olivier Grisel



:probab1.



CONTENTS

GOSIM Paris 2026

Part 01

Adopting the Array API Standard in scikit-learn

Part 02

Implementation and Challenges

Part 03

Demo



01

Adopting the Array API Standard in scikit-learn



- scikit-learn grew around **NumPy-style arrays on the CPU**
- many workflows now use **GPUs** and **other array libraries**
- support is **experimental** and has grown since **1.3** (June 2023):
more estimators and functions over time



- scikit-learn is an array consuming library:
 - inputs are arrays
 - outputs are arrays
- goal:
let serious work **stay on the hardware and library** the data already lives in
- users can bring **CuPy** or **PyTorch** data today; **DPNP** is next and **JAX** is being considered



Consortium for Python Data API Standards maintains the **Array API** standard (specification published at data-apis.org)

- **array-api-tests**: shared conformance test suite array producing libraries
- **array-api-strict**: strict, minimal implementation used heavily in scikit-learn CI
- **array-api-compat**: bridges NumPy, PyTorch, and CuPy where their APIs differ from Array API standard; vendored in scikit-learn since 1.7
- **array-api-extra**: helpers for operations the core standard does not define; vendored in scikit-learn since 1.7



Array API in scikit-learn is **opt-in**:

- **SCIPY_ARRAY_API=1** environment variable to enable it in **SciPy**
- scikit-learn global configuration (`set_config`) or the `config_context` context manager: **array_api_dispatch=True**
- scikit-learn user docs: “**Array API support (experimental)**”





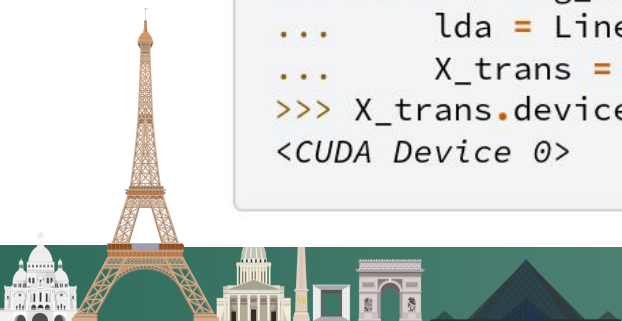
12.2.2. Example usage

The example code snippet below demonstrates how to use [CuPy](#) to run `LinearDiscriminantAnalysis` on a GPU:

```
>>> from sklearn.datasets import make_classification
>>> from sklearn import config_context
>>> from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
>>> import cupy

>>> X_np, y_np = make_classification(random_state=0)
>>> X_cu = cupy.asarray(X_np)
>>> y_cu = cupy.asarray(y_np)
>>> X_cu.device
<CUDA Device 0>

>>> with config_context(array_api_dispatch=True):
...     lda = LinearDiscriminantAnalysis()
...     X_trans = lda.fit_transform(X_cu, y_cu)
>>> X_trans.device
<CUDA Device 0>
```



02

Implementation and Challenges



[Code](#) [Issues 1.5k](#) [Pull requests 528](#) [Agents](#) [Discussions](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#)

Path for Adopting the Array API spec #22352

Closed



thomasjpfan opened on Feb 1, 2022 · edited by thomasjpfan Edits Member ...

I have been experimenting with adopting the [Array API](#) spec into scikit-learn. The Array API is one way for scikit-learn to run on other hardware such as GPUs.

I have some POCs on my fork for [LinearDiscriminantAnalysis](#) and [GaussianMixture](#). Overall, there is runtime performance benefit when running on CuPy compared to NumPy, as shown in notebooks for [LDA \(14x improvement\)](#) and [GMM \(7x improvement\)](#).

Official acceptance of the Array API in numpy is tracked as [NEP 47](#).

Proposed User API

Here is the proposed API for dispatching. We require the array to adopt the [Array API standard](#) and we have a configuration option to turn on Array API dispatching:



Computations stay on the same **namespace** and **device** as the input arrays:

```
from sklearn.utils._array_api import get_namespace, device

xp, is_array_api = get_namespace(X)  # same library as `X`
device = device(X)                  # same
device as `X`
```

Inside estimators:

1. keep operations on namespace xp
2. create new arrays on namespace xp and device



Array API standard < NumPy

- scikit-learn patches gaps with internal helpers:

```
def nanmin(X, axis=None, xp=None):  
    # TODO: refactor once nan-aware reductions are standardized:  
    # https://github.com/data-apis/array-api/issues/621  
    xp, _, device_ = get_namespace_and_device(X, xp=xp)  
    if _is_numpy_namespace(xp):  
        return xp.asarray(numpy.nanmin(X, axis=axis))  
  
    else:  
        mask = xp.isnan(X)  
        X = xp.min(  
            xp.where(mask, xp.asarray(+xp.inf, dtype=X.dtype, device=device_), X),  
            axis=axis,  
        )  
        # Replace Infs from all NaN slices with NaN again  
        mask = xp.all(mask, axis=axis)  
        if xp.any(mask):  
            X = xp.where(mask, xp.asarray(xp.nan, dtype=X.dtype, device=device_), X)  
        return X
```



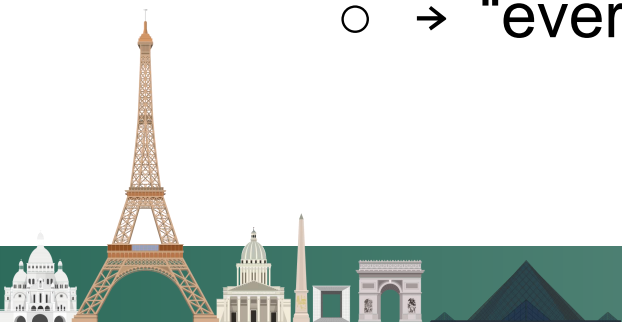
Common in productionised ML: **input arrays that differ or change by step**

- train models on **GPU** then run inference and deploy on **CPU**
- `fit`, `predict`, `transform` or a metric function may be called with new data from a **different array library or device**
- **string arrays** (`labels`) in classification (NumPy only)



Common in productionised ML: **input arrays that differ or change by step**

- explicitly **support flexible models** by adding a helper (`convert_estimator`) that moves fitted estimator attributes to the namespace and device of the array passed as `X` to `predict`
- explicitly **support mixed array input** in single calls
 - → "everything follows `X`" (in estimators)
 - → "everything follows `y_pred / y_score`" in scoring functions



“everything follows X ”

```
from functools import partial
from sklearn.linear_model import RidgeClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import FunctionTransformer, TargetEncoder

pipeline = make_pipeline(
    TargetEncoder(),
    # Convert feature array `X` to Torch CUDA device
    FunctionTransformer(partial(torch.asarray, dtype="float32", device="cuda")),
    RidgeClassifier(solver="svd"),
)
```



Computation intense tasks on **compiled code** (Cython, C, C++) optimised for CPU is an ongoing challenge. Estimators can:

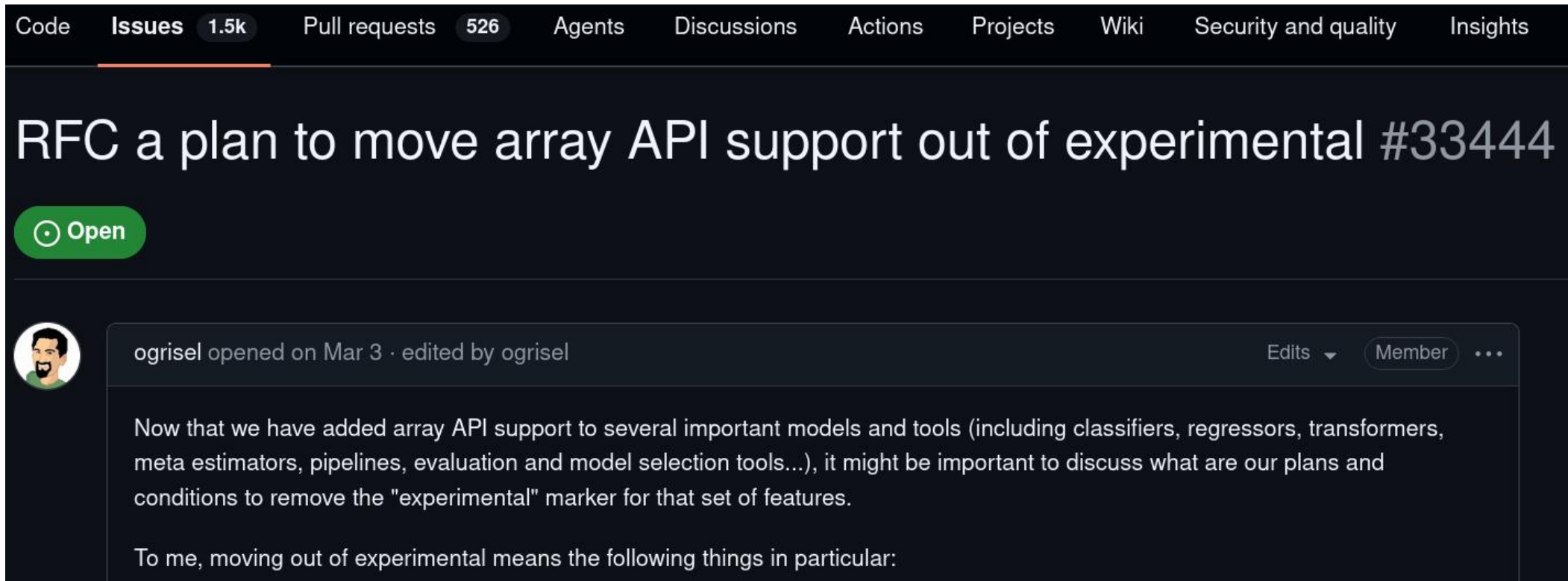
- a) use the compiled path by copying data to CPU, then **map results back** to the input namespace and device
- b) use the compiled path only for NumPy inputs; otherwise follow an Array API Python path



- trade-offs between runtime **speed** and **memory** footprint
- maintainers often need to compare and **profile multiple options**
- benchmarking on realistic workloads versus **edge cases**
- clearly **define rules** for (silent) conversions to prevent surprises for users



- plans of moving out of experimental mode
- stable Array API behaviour in scikit-learn requires **predictable behaviour** in upstream implementations of Array API, especially in **SciPy**



The screenshot shows a GitHub issue page for the repository 'scikit-learn'. The issue title is 'RFC a plan to move array API support out of experimental #33444'. It is marked as 'Open' with a green button. The issue was opened by 'ogrisel' on March 3 and edited by 'ogrisel'. The content of the issue discusses the plan to move array API support out of experimental mode, mentioning that it has been added to several important models and tools (including classifiers, regressors, transformers, meta estimators, pipelines, evaluation and model selection tools...). The issue also mentions that it might be important to discuss what are the plans and conditions to remove the "experimental" marker for that set of features. The issue is currently open and has 1.5k issues.

Code **Issues** 1.5k Pull requests 526 Agents Discussions Actions Projects Wiki Security and quality Insights

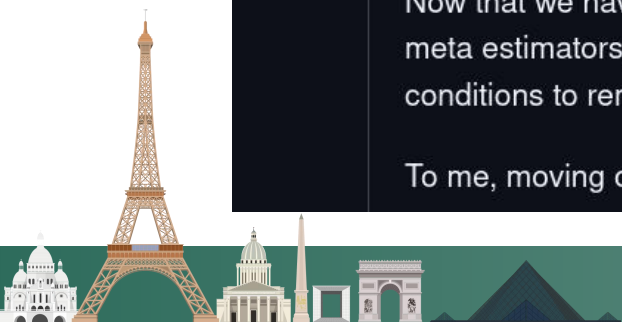
RFC a plan to move array API support out of experimental #33444

Open

 ogrisel opened on Mar 3 · edited by ogrisel Edits Member ...

Now that we have added array API support to several important models and tools (including classifiers, regressors, transformers, meta estimators, pipelines, evaluation and model selection tools...), it might be important to discuss what are our plans and conditions to remove the "experimental" marker for that set of features.

To me, moving out of experimental means the following things in particular:



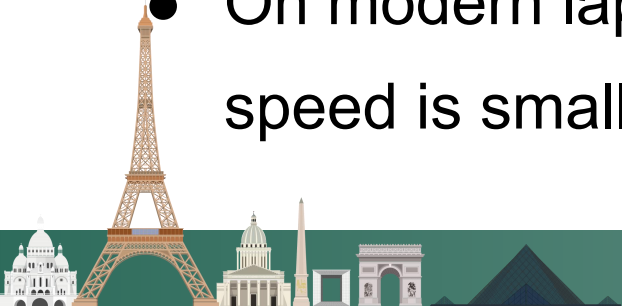
03

Demo

https://github.com/ogrisel/notebooks/blob/master/poly_reg_array_api.ipynb



- The same code can run:
 - with NumPy or PyTorch on any CPU
 - with CuPy or PyTorch CUDA on NVIDIA GPUs
 - with PyTorch MPS on Apple GPUs
 - with DPNP or PyTorch XPU on Intel GPUs
 - and probably more (to be tested)
- On Google Colab, using the CPU is now very good: using a GPU can speed-up cross-validation or grid search by a factor of 15x to 30x.
- On modern laptops (e.g. Intel or Apple) the difference between CPU and GPU speed is smaller (1.5x to 5x) but still nice!

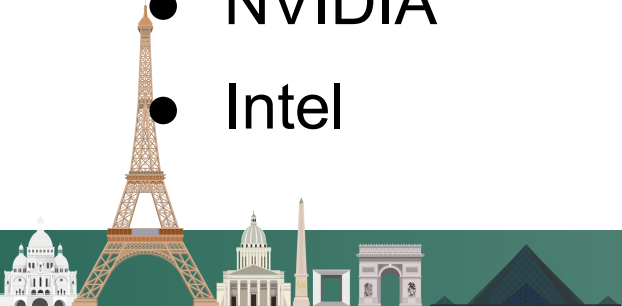


Array API support in scikit-learn is a multi-year effort.

Many individual contributors and organizations have made this possible.

This work was/is supported by the following organizations:

- Quansight
- Probabl
- NASA ROSES Grants 80NSSC25K7215 and 80NSSC22K0405
- NVIDIA
- Intel



Thanks!



:probabl.

https://scikit-learn.org/dev/modules/array_api.html

<https://blog.scikit-learn.org/updates/update-array-api/>

These slides:

<https://docs.google.com/presentation/d/1kHdH0VW0zcJ776RfuQeSsC0XB2EfQ1Ej>

